



> Menu

Overview

Relevant source files

Purpose: This document provides a high-level introduction to YouRAM, explaining its architecture, core abstractions, and design flow. It serves as the entry point for understanding the codebase structure and relationships between major subsystems.

YouRAM is an **SRAM compiler** implemented in Rust that generates Static Random-Access Memory circuits from user specifications. The system takes memory configuration parameters (word count, word width, process technology) and produces complete SRAM designs that can be simulated, characterized, and exported to industry-standard formats including SPICE netlists, Verilog HDL, and Liberty timing libraries.

For detailed information on specific subsystems:

- Circuit representation abstractions: see [Core Circuit Architecture](#)
- SRAM module hierarchy: see [Memory Hierarchy](#)
- Technology abstraction: see [Process Design Kit \(PDK\) Integration](#)
- Simulation capabilities: see [Simulation System](#)
- Output format generation: see [Export System](#)

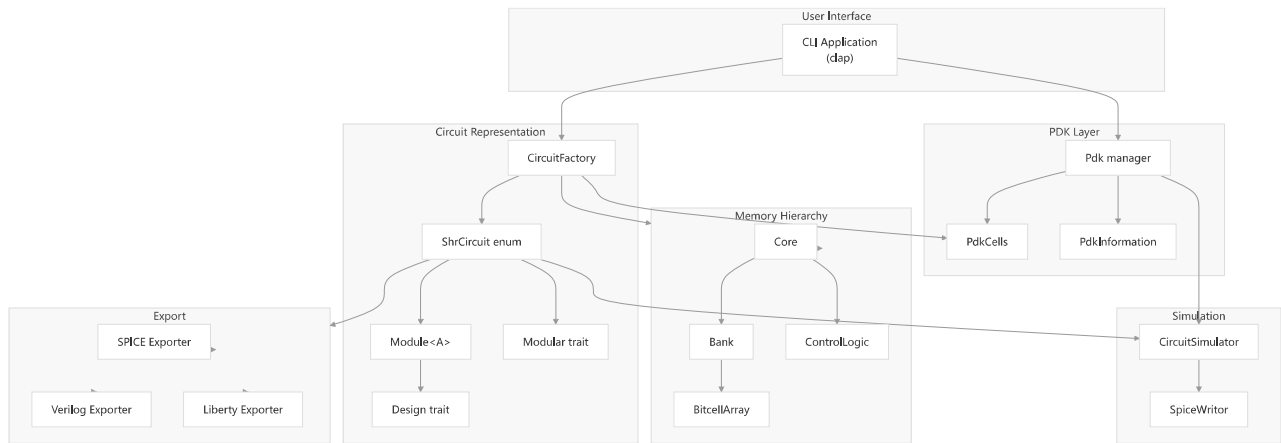
Sources: `src/circuit/mod.rs` | 1-116 `Cargo.toml` | 1-26

System Architecture

YouRAM implements a **layered architecture** with five primary subsystems that operate on a shared circuit representation. The design follows a clear separation of concerns, enabling independent development and testing of each layer.

Ask Devin about MoleSir/YouRAM





Sources: `src/circuit/mod.rs` | 1-116

Core Abstractions

The following table maps high-level concepts to their code implementations:

Concept	Code Entity	Location	Purpose
Circuit Design	<code>ShrCircuit</code> enum	<code>src/circuit/mod.rs</code> 33-36	Union type representing either <code>Module</code> or <code>Primitive</code> circuits
Design Interface	<code>Design</code> trait	<code>src/circuit/mod.rs</code> 19-30	Common interface for accessing name and ports
Hierarchical Module	<code>Module<A></code> struct	Referenced in <code>mod.rs</code>	Parameterized circuit module with instances and nets
Module Behavior	<code>Modular</code> trait	Referenced in <code>mod.rs</code>	Trait for circuit modules supporting hierarchical design
Component Factory	<code>CircuitFactory</code>	<code>src/circuit/factory.rs</code>	Constructs circuit components with PDK integration
Shared Ownership	<code>Shr<T></code> type alias	<code>src/circuit/shared.rs</code>	<code>Arc<RwLock<T>></code> wrapper for shared mutable state
String Pooling	<code>ShrString</code> struct	<code>src/circuit/srdstring.rs</code>	Interned strings for memory efficiency

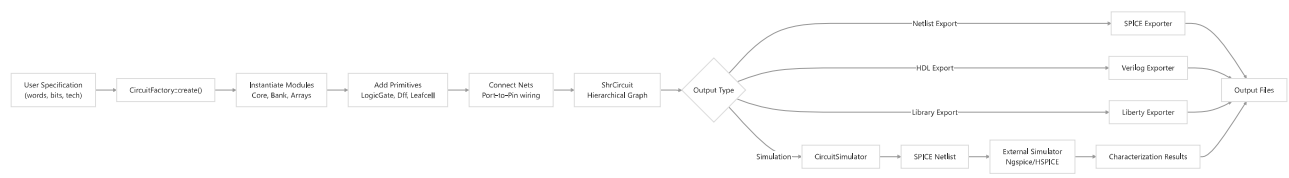
Ask Devin about MoleSir/YouRAM



Design Flow

YouRAM follows a three-phase pipeline: **Construction** → **Representation** → **Output Generation**. Each phase operates on the circuit graph data structure built in the previous phase.

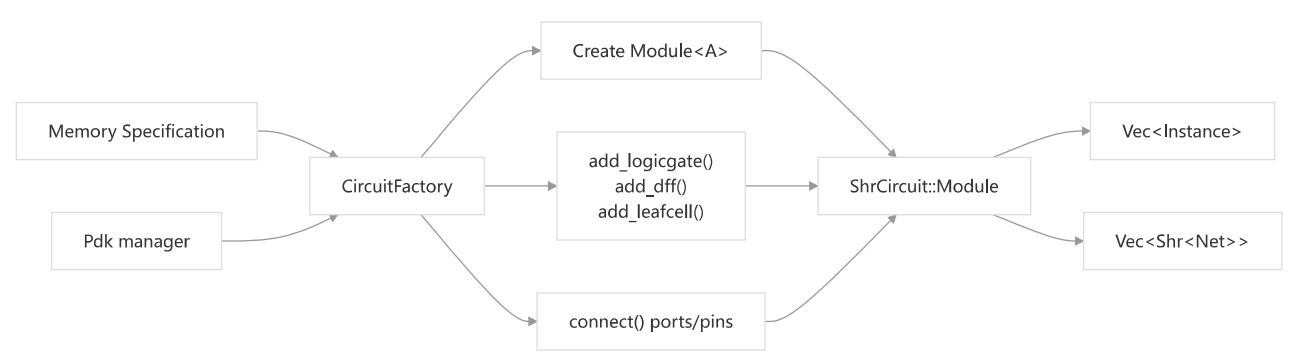
End-to-End Circuit Generation Flow



Sources: Based on system architecture diagrams

Construction Phase

The `circuitFactory` serves as the central orchestrator for building circuit components. It queries the PDK layer for standard cells and leaf cells, instantiates memory modules with specified parameters, and wires ports to form the complete circuit graph.



Sources: `src/circuit/mod.rs` | 33-67

Representation Phase

The internal representation uses `shr_circuit` as a unified type that wraps either hierarchical modules (`Shr<dyn Modular>`) or primitive components (`Shr<dyn Primitive>`). This enum enables polymorphic traversal during export and simulation.

Key implementation details:

Ask Devin about MoleSir/YouRAM



- Shared ownership through `shr<T>` enables graph-like references

Sources: `src/circuit/mod.rs` | 33-115

Technology Abstraction

YouRAM abstracts process technology through the PDK (Process Design Kit) layer, enabling support for multiple fabrication technologies without modifying memory generation logic.

PDK Integration Pattern

PDK Component	Purpose	Data Source
<code>PdkConfig</code>	File paths to technology files	Configuration file
<code>PdkInformation</code>	Timing parameters, voltage levels	Liberty (.lib) files
<code>PdkCells</code>	Component definitions and netlists	SPICE (.sp) and CDL (.cdl) files
<code>PdkModels</code>	Transistor models (NMOS/PMOS)	Model files (.pm)

The `CircuitFactory` queries `PdkCells` during component instantiation, while `CircuitSimulator` includes model files from `PdkConfig` in generated SPICE netlists. This indirection allows the same memory generation code to target different process nodes.

Sources: Based on PDK integration diagrams, `Cargo.lock` | 524-535 (reda-lib dependency)

Key Dependencies

YouRAM relies on several external crates that provide core functionality:

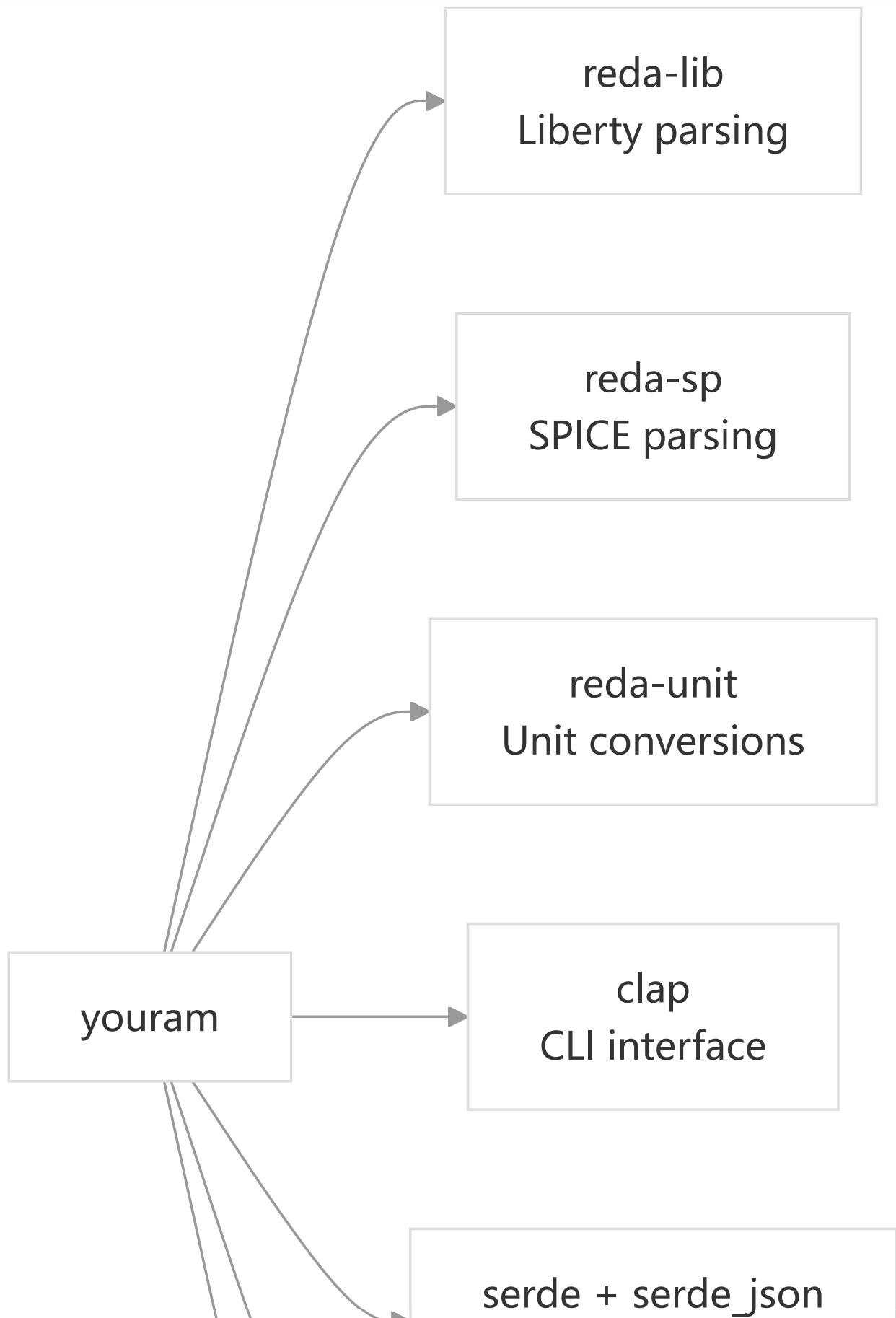
Primary Dependencies

Ask Devin about MoleSir/YouRAM



Ask Devin about MoleSir/YouRAM





Ask Devin about MoleSir/YouRAM



```
graph LR; A[ ] --> B[thiserror Error handling]; A --> C[tracing Logging];
```

thiserror
Error handling

tracing
Logging

Sources: [Cargo.toml](#) | 6-24 | [Cargo.lock](#) | 524-585

Reda Ecosystem

The **reda ecosystem** provides industry-standard EDA file format support:

- **reda-lib** (v0.7.0): Liberty (.lib) file parsing for timing/power information
- **reda-sp** (v0.2.0): SPICE netlist parsing and generation
- **reda-unit** (v0.3.0): Physical unit handling (voltage, time, capacitance)

These dependencies parse external PDK files and enable YouRAM to read standard cell libraries and write simulation netlists.

Sources: [Cargo.lock](#) | 524-585

Error Handling Architecture

YouRAM implements centralized error handling through the `YouRAMError` enumeration, which aggregates domain-specific errors from all subsystems.

Ask Devin about MoleSir/YouRAM



Error Type	Source Module	Purpose
YouRAMError	src/circuit/error.rs	Top-level error aggregation
CircuitError	Circuit operations	Module/primitive construction failures
SimulateError	Simulation system	SPICE generation and execution errors
CharzError	Characterization	Measurement parsing failures
PdkError	PDK layer	Technology file loading errors

The `ErrorContext` trait (referenced in error module) enables **error enrichment**, allowing functions to add contextual information while propagating errors up the call stack.

Sources: `src/circuit/mod.rs` | 6 (error module reference)

Memory Efficiency Utilities

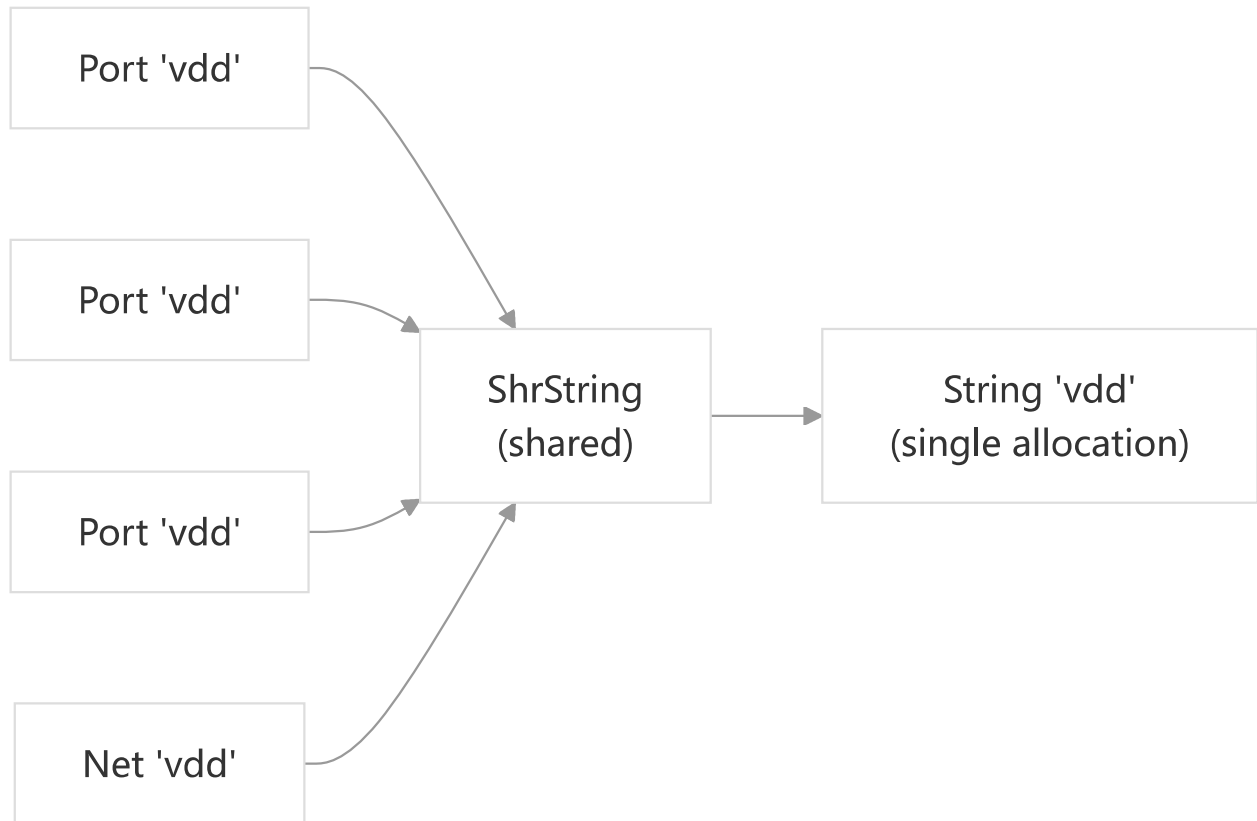
Two cross-cutting utilities optimize memory usage in the circuit representation:

ShrString: String Interning

`shrString` implements **string pooling** to reduce memory overhead for frequently repeated identifiers (port names, net names, module names). Instead of storing duplicate strings, it maintains a shared reference to a single allocation.

Ask Devin about MoleSir/YouRAM





Sources: `src/circuit/mod.rs` | 12

Shr: Shared Mutable State

`Shr<T>` wraps `Arc<RwLock<T>>` to provide **shared ownership with interior mutability**. This enables the graph-like circuit structure where multiple instances reference the same ports and nets, allowing bidirectional traversal.

Sources: `src/circuit/mod.rs` | 11

Output Capabilities

YouRAM generates multiple output formats from a single circuit representation, supporting various stages of the ASIC design flow:

Ask Devin about MoleSir/YouRAM



Format	File Extension	Purpose	Downstream Tools
Verilog HDL	.v	Logic synthesis	Synopsys DC, Cadence Genus
Liberty Library	.lib	Timing/power characterization	STA tools, Place & Route

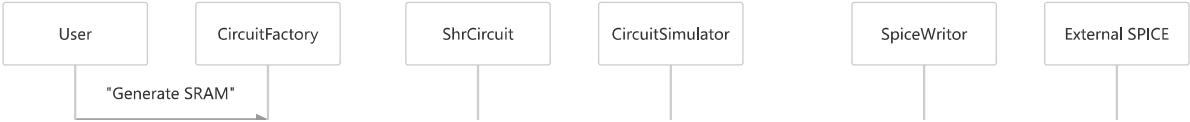
Each exporter traverses the `shrcircuit` hierarchy and generates format-specific output. The SPICE exporter handles subcircuit definitions, port ordering, and instance flattening. The Verilog exporter produces synthesizable RTL with module declarations. The Liberty exporter includes timing arcs, power consumption, and area information.

Sources: Based on export system diagrams

Simulation Workflow

The simulation system generates SPICE netlists with stimuli and measurements, executes external simulators, and parses results for characterization.

Simulation Pipeline



Ask Devin about MoleSir/YouRAM

